

Name: Koda Adam

PROJECT DELIVERABLE

Intelligent Vendor Routing Platform

One API · Many Vendors · Zero Juggling

A middleware platform that routes every request to the best third-party vendor — based on cost, latency, priority and live health — with automatic failover and AI explanations.

Source Code	github.com/Adamkoda2306/Intelligent-Vendor-Routing-Platform
Live Frontend	intelligent-vendor-routing-platform.vercel.app
Live Backend API	intelligent-vendor-routing-platform.onrender.com
Stack	Node.js · TypeScript · Express · MongoDB · Gemini 2.5 Flash · Chart.js

1. Project Links & Documentation

Everything about this project is publicly accessible. The repository contains the complete source code, an automated test suite, CI/CD workflows, and a documentation set where every major component has its own detailed guide.

Live deployments & source

Resource	Link
GitHub repository (source code)	https://github.com/Adamkoda2306/Intelligent-Vendor-Routing-Platform
Deployed frontend (Vercel)	https://intelligent-vendor-routing-platform.vercel.app/index.html
Deployed backend API (Render)	https://intelligent-vendor-routing-platform.onrender.com/

Note: the backend runs on Render's free tier and sleeps when idle — the first request after inactivity may take 30–60 seconds to wake the service.

Documentation (README files)

Document	What it covers
README.md (root)	Full project overview, features, architecture, quick start
backend/README.md	Backend architecture, setup, environment variables, running the API
frontend/README.md	Every dashboard page, the API client, configuration & deployment
docs/API.md	Full API reference — every endpoint with request/response examples & cURL
docs/ROUTING_ENGINE.md	Deep-dive into all 5 routing strategies, algorithms and edge cases
docs/TESTING.md	Unit, integration & coverage testing guide
docs/CICD-PIPELINE.md	GitHub Actions pipelines, test gating, Render/Vercel deploy hooks

2. Project Overview

Modern applications rarely rely on a single vendor for a capability such as PAN verification, OCR, SMS or KYC. Each vendor differs in cost, latency, reliability and rate limits, and the application ends up juggling multiple SDKs, retry logic and failover rules.

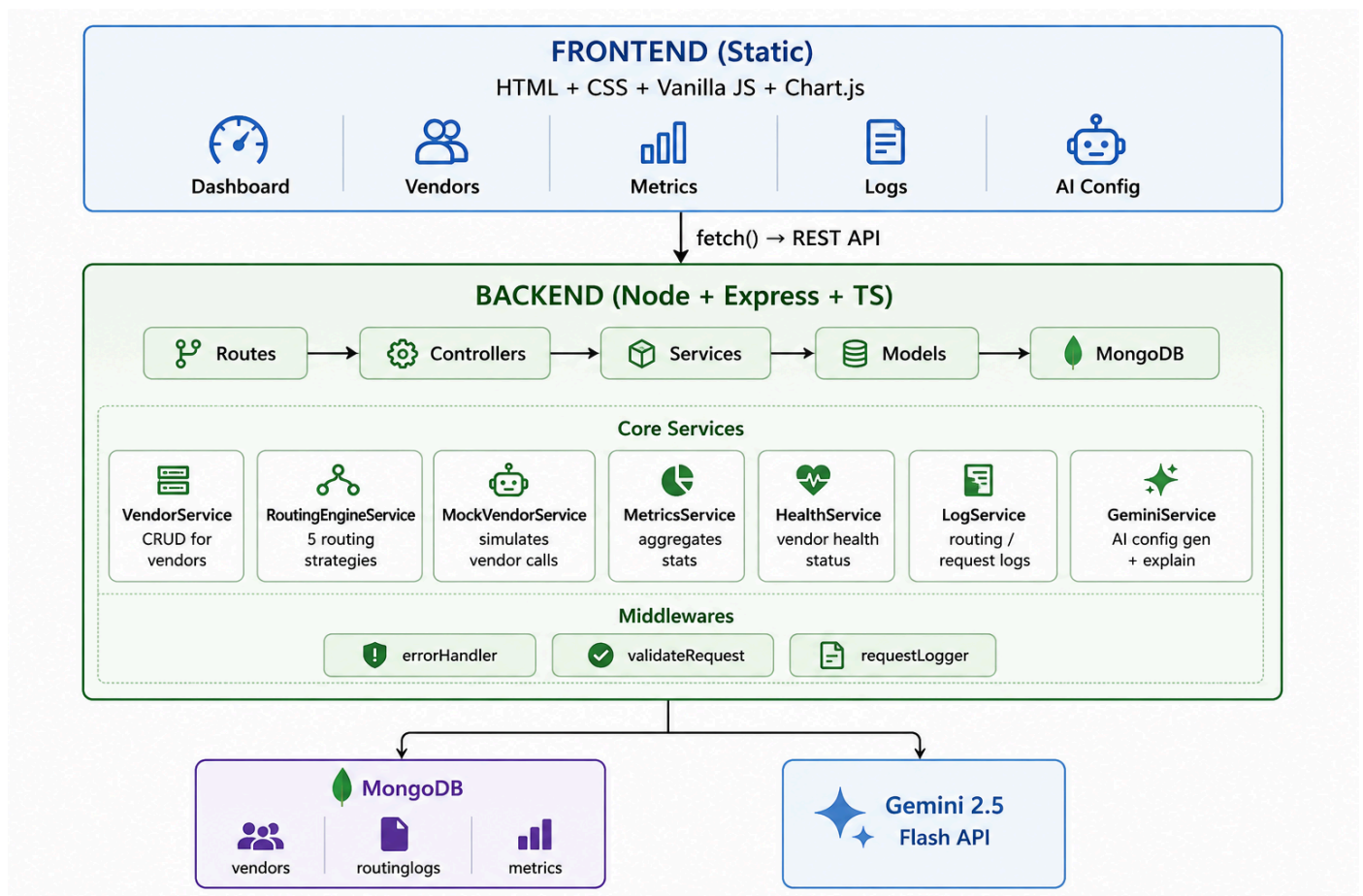
This platform replaces all of that with a single call: **POST /route**. The client describes the capability it needs and (optionally) its requirements; the platform does the rest:

- Filters vendors that support the capability, are enabled, and are not OFFLINE
- Applies one of five routing strategies to pick the best vendor
- Calls a simulated vendor with randomized latency and randomized failures
- Automatically fails over to the next-best vendor if the chosen one fails
- Records metrics, re-evaluates vendor health, and persists a decision log
- Uses Gemini 2.5 Flash to generate routing configs from plain English and to explain any routing decision

Scope: intentionally basic-to-intermediate — no authentication, microservices, Docker, Redis or Kafka — so the routing logic remains readable and demonstrable for a final-year project.

3. System Architecture

The system is a classic three-tier architecture with an AI sidecar. A fully static frontend talks to a layered Express backend (routes → controllers → services → models), which persists to MongoDB and calls the Gemini API for the AI features. Controllers never touch the database directly — all business logic and data access live in the service layer.



4. Routing Flow — POST /route

The diagram below traces a single request through the platform: validation, capability filtering, health filtering, strategy selection based on the client's requirements, the (simulated) vendor call, and the failover loop that retries the next vendor until one succeeds or the pool is exhausted. Both terminal paths — success and total failure — end with a persisted log and updated metrics.

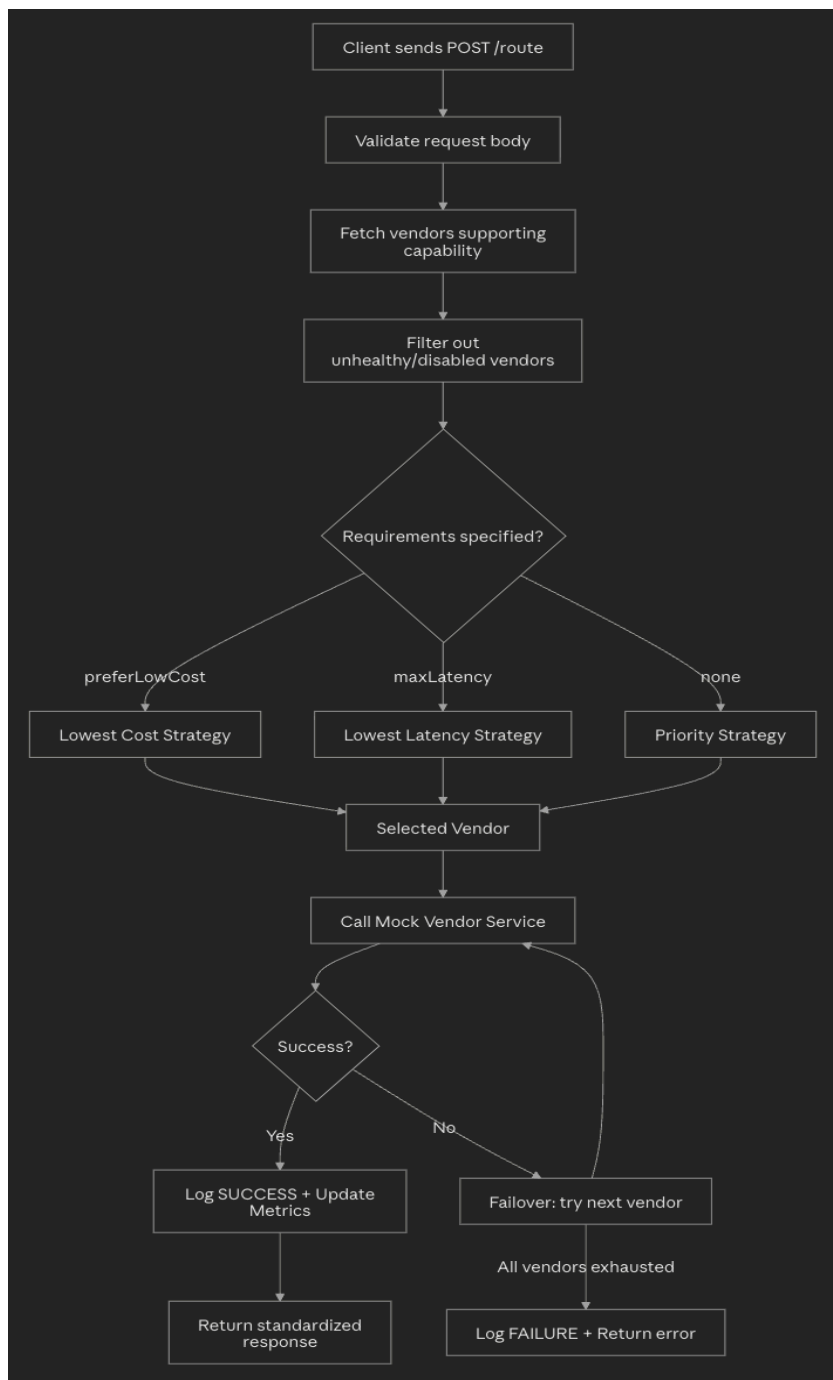


Figure 1 — End-to-end routing decision flow with automatic failover

5. Vendor Routing Design — The 5 Strategies

The routing engine (`routingEngine.service.ts`) is a pure, stateless module: no I/O, no shared state. Given the pre-filtered pool of eligible vendors and the client's requirements, it returns the selected vendor together with a human-readable reason explaining the decision.

Strategy	Selection rule	Best suited for
PRIORITY	Lowest priority number wins (rank 1 beats rank 2). Default strategy.	Reliability-first routing where an operator has hand-ranked vendors
WEIGHTED	Roulette-wheel random pick; probability proportional to weight (70/20/10 → ~70%/20%/10% of traffic)	Traffic splitting, canary rollouts, contractual volume commitments
LOWEST_COST	Smallest cost per request wins	High-volume, non-urgent workloads (bulk verification, batch OCR)
LOWEST_LATENCY	Smallest <code>avgLatencyMs</code> wins; optional <code>maxLatency</code> ceiling filters slower vendors first (soft constraint — falls back to full pool if it empties)	User-facing, time-critical flows (signup KYC, OTP delivery)
FAILOVER	Automatic, never client-selected: after a failure, picks the highest-priority vendor not yet attempted	The always-on safety net behind every other strategy

Strategy resolution order

When a request arrives, the engine resolves the strategy with strict precedence — an explicit strategy always wins, requirement flags are consulted next, and **PRIORITY** is the final default:

```
requirements.strategy  ("PRIORITY" | "WEIGHTED" | "LOWEST_COST" | "LOWEST_LATENCY")
  ↓ (not given)
requirements.preferLowCost    → LOWEST_COST
requirements.preferLowLatency → LOWEST_LATENCY
requirements.maxLatency alone → LOWEST_LATENCY (with ceiling)
  ↓ (nothing specified)
DEFAULT                      → PRIORITY
```

Every selection carries a concrete, numbers-included reason string (e.g. *“Selected Vendor C because it has the lowest cost (cost=2) among eligible vendors.”*) — this powers both the routing logs and the AI explanation feature. The complete algorithms, tie-breaking, and edge cases are documented in docs/ROUTING_ENGINE.md.

6. Failover Handling

Failover is a behavior, not a client-selectable strategy — it activates automatically whenever a chosen vendor's call fails, regardless of which strategy made the original selection. After a failure, the goal shifts from optimizing the client's preference to maximizing the chance of any success, so failover always retries in priority order among the vendors not yet attempted.

```
Attempt 1 : PRIORITY selects Vendor A (priority 1)      → call FAILS
Attempt 2 : FAILOVER  - attempted={A}, picks Vendor B  → call FAILS
Attempt 3 : FAILOVER  - attempted={A,B}, picks Vendor C → call SUCCEEDS

Result    : 200 OK · strategyUsed="FAILOVER" · vendorSelected="Vendor C"
           attemptedVendors=["Vendor A", "Vendor B", "Vendor C"]
```

- Termination is guaranteed — every attempt excludes one more vendor from the finite pool
- Metrics and health are recorded for every attempted vendor, including the failed ones
- If all vendors fail, the API returns 502 with a FAILED log listing every attempt
- The client never orchestrates any of this — one request in, one response out

7. Metrics Tracking & Health Monitoring

A rolling metrics document is maintained per vendor and updated after every routed request (success or failure): total / successful / failed requests, cumulative and average latency, error rate, and availability. Aggregated platform-wide stats plus the per-vendor breakdown are exposed at `GET /vendor-metrics` and visualized on the dashboard with Chart.js.

Automatic health classification

After every request, the attempted vendor's health is re-evaluated from its rolling error rate (a vendor needs at least 5 recorded requests before being judged):

Status	Error rate	Effect on routing
HEALTHY	below 20%	Fully eligible
WARNING	20% – 50%	Still eligible, flagged on the dashboard
OFFLINE	50% and above	Excluded from routing entirely

This closes the loop: an unreliable vendor degrades its own health through real traffic, gets excluded automatically, and routing shifts to the remaining vendors — no manual intervention required.

8. Explanation of Routing Decisions

Explainability is designed in at two layers, so every decision the platform makes can be audited and understood — by engineers and by non-technical stakeholders.

Layer 1 — Engine-generated reasons (deterministic)

Every strategy writes a concrete, numbers-included reason at selection time. It is returned in the API response and persisted with the routing log:

- "Selected Vendor A because it has the highest priority (priority=1) among eligible vendors."
- "Selected Vendor B via weighted random distribution (weight=30/110)."
- "Selected Vendor C because it has the lowest cost (cost=2) among eligible vendors."
- "Failover triggered. Selected next available vendor Vendor B (priority=2)."

Layer 2 — AI explanations for stakeholders (Gemini)

On the Routing Logs page, every row has an **Explain** button. It sends the log's ID to `POST /ai/explain-route`; the backend fetches the full log, hands it to Gemini 2.5 Flash, and returns a plain-English narrative:

```
"Vendor A was tried first but failed to process the request, so the platform automatically moved to the next available vendor, Vendor B, which handled it successfully in 612ms. This failover behavior ensures the client still gets a response even when the top-priority vendor has an outage."
```

Because the AI explanation is grounded in the persisted log — strategy, reason, attempts, latency, status — it narrates what actually happened rather than speculating.

9. Rule & Configuration Design

Routing behavior is fully data-driven. Each vendor document carries the parameters every strategy reads, so behavior is changed by editing vendor records (via the dashboard or API) — never by touching code:

Field	Default	Role in routing
priority	10	Rank for PRIORITY and FAILOVER — lower number = higher priority
weight	50	Relative traffic share (0–100) for WEIGHTED; 0 drains traffic
cost	—	Per-request cost compared by LOWEST_COST
avgLatencyMs	—	Expected latency compared by LOWEST_LATENCY
capabilities	—	Which request types the vendor can serve
enabled	true	Kill-switch — disabled vendors are never routed to
failureRate	0.05	Probability (0–1) the mock vendor simulates a failure
rateLimitPerMin	60	Stored for future enforcement

Per-request rules

Clients steer individual requests through the optional requirements object — strategy, preferLowCost, preferLowLatency, and maxLatency (a soft latency ceiling) — resolved by the precedence cascade shown in Section 5.

AI-generated configuration (Gemini)

An operator can express a policy in plain English and `POST /ai/generate-config` converts it into structured JSON:

```
Instruction : "Use Vendor A for 70% traffic. Vendor B for 30%.  
             If latency exceeds 2 seconds use Vendor C."  
Generated   : {  
    "strategy": "WEIGHTED",  
    "vendorWeights": { "Vendor A": 70, "Vendor B": 30 },  
    "maxLatencyMs": 2000,  
    "notes": "Traffic split 70/30; fall back to Vendor C past 2000ms."  
}
```

10. Sample Vendor Configurations (Seed Data)

The repository ships with a seed script (`src/Utils/seedVendors.ts`, run with `npm run seed`) that inserts three deliberately contrasting vendors. Their trade-offs are what make every strategy demonstrable: each strategy picks a different vendor for the same request.

Vendor	Priority	Weight	Cost	Avg Latency	Profile
Vendor A	1	70	10	300 ms	Fast · costly · reliable (2% failures)
Vendor B	2	30	6	700 ms	Balanced (5% failures)
Vendor C	3	10	2	1500 ms	Cheap · slow · flaky (20% failures)

Capabilities — Vendor A: PAN_VERIFICATION, OCR, KYC · Vendor B: PAN_VERIFICATION, OCR, SMS · Vendor C: PAN_VERIFICATION, SMS, KYC. All three start HEALTHY, enabled, with rate limits of 100 / 80 / 50 requests per minute respectively.

Full seed document — Vendor A (JSON)

```
{
  "name": "Vendor A",
  "priority": 1,
  "weight": 70,
  "cost": 10,
  "avgLatencyMs": 300,
  "rateLimitPerMin": 100,
  "capabilities": ["PAN_VERIFICATION", "OCR", "KYC"],
  "healthStatus": "HEALTHY",
  "enabled": true,
  "failureRate": 0.02
}
```

The seed script connects to MongoDB, clears the vendors collection, inserts all three documents, and disconnects — giving every fresh environment an identical, demo-ready starting state. Consequence of the trade-offs: PRIORITY and LOWEST_LATENCY pick Vendor A, LOWEST_COST picks Vendor C, and WEIGHTED splits traffic $\approx 70/30/10$ — while Vendor C's 20% failure rate regularly exercises the failover path and the health downgrade to WARNING.

11. API Design

The API is a small, consistent REST surface mounted under `/api/v1 (versioning)`. Every response — success or error — uses the same envelope: `{ "success", "message", "data" }` or `{ "success": false, "error" }`. Full request/response documentation with cURL examples lives in [docs/API.md](#).

Method	Endpoint	Purpose
POST	/vendors	Create a vendor

Method	Endpoint	Purpose
GET	/vendors	List all vendors (sorted by priority)
PUT	/vendors/:id	Partial update of a vendor
DELETE	/vendors/:id	Delete a vendor
POST	/route	Route a request to the best available vendor (core endpoint)
GET	/vendor-metrics	Aggregated + per-vendor metrics
GET	/routing-logs?limit=	Recent routing decisions, newest first
GET	/health	Health classification of every vendor
POST	/ai/generate-config	Plain English → routing config JSON (Gemini)
POST	/ai/explain-route	Plain-English explanation of a routing decision (Gemini)

Sample request — POST /route

```
POST https://intelligent-vendor-routing-platform.onrender.com/api/v1/route
Content-Type: application/json
{
  "capability": "PAN_VERIFICATION",
  "payload": { "pan": "ABCDE1234F", "name": "Rahul Sharma" },
  "requirements": { "preferLowCost": true }
}
```

Sample response — success after automatic failover (200 OK)

```
{
  "success": true,
  "message": "Request routed successfully",
  "data": {
    "strategyUsed": "FAILOVER",
    "vendorSelected": "Vendor B",
    "attemptedVendors": ["Vendor C", "Vendor B"],
    "reason": "Failover triggered. Selected next available vendor Vendor B (priority=2).",
    "latencyMs": 612,
    "status": "SUCCESS",
    "errorMessage": null,
    "responsePayload": {
      "vendor": "Vendor B",
      "capability": "PAN_VERIFICATION",
      "verified": true,
      "echo": { "pan": "ABCDE1234F", "name": "Rahul Sharma" },
      "processedAt": "2026-07-04T10:16:44.812Z"
    }
  }
}
```

Error semantics: 400 invalid input · 404 unknown resource / no eligible vendor · 502 all vendors failed or Gemini unavailable · 500 unexpected error — all in the same envelope.

12. Code Testing & CI/CD

- TypeScript end-to-end with shared type definitions (routing, vendor, API types)
- Strict layering: routes → controllers → services → models; controllers never touch the database
- The routing engine is pure and stateless — trivially testable
- Centralized error handling, request logging and response envelope helpers
- Configuration isolated in a typed env loader; secrets never hardcoded

Testing (Jest · ts-jest · Supertest · mongodb-memory-server)

- **Unit tests** — every service in isolation with all dependencies mocked: all 5 strategies, failover exhaustion, metrics math, health thresholds at exact boundaries, Gemini JSON parsing
- **Integration tests** — real HTTP via Supertest against an in-memory MongoDB: vendor CRUD, /route incl. forced failover chains and the all-vendors-fail path, observability and AI endpoints
- **Deterministic randomness** — Math.random and vendor-call outcomes are mocked/scripted, so the randomized simulator never causes flaky tests
- **Coverage** — full-suite coverage report generated locally and published as a CI artifact

CI/CD (GitHub Actions)

Every push touching backend/** runs three parallel jobs — unit tests, integration tests, coverage. Only when all three pass on main does the pipeline trigger the Render deploy hook; the frontend deploys to Vercel the same way on frontend/** changes. Broken code can be pushed, but it can never reach production.